

Federated Learning for Aircraft Engine Health Monitoring Using C-MAPSS

Submitted By

Mansura Naznine

Research Assistant, Qatar University

MSc Student, Universiti Kebangsaan Malaysia

Submitted To

Shafika Showkat Moni

Assistant Professor, Electrical Engineering and Computer Science Dept.

Daytona College of Engineering,

Embry-Riddle Aeronautical University

Github Repository: <https://github.com/naznine/federated-aircraft-health-monitoring>

Date: 1 July, 2026

1. Introduction

Aircraft engine health monitoring is important for predicting failures early and planning maintenance safely. In this project, I built a federated learning system using the C-MAPSS turbofan engine dataset [1]. The main goal was to predict two outputs: Remaining Useful Life (RUL) and early fault risk. In the federated setup, the four C-MAPSS subsets, FD001, FD002, FD003, and FD004, were treated as four different airline clients. Each client trained locally on its own data, and a central server combined the model weights using FedAvg. This setup allows the model to learn from multiple clients without sharing raw sensor data. I completed the main federated learning baseline and then explored three research questions: RQ2 on class imbalance across clients, RQ4 on concept drift, and RQ5 on validation bias under Non-IID client data.

Key Contributions:

1. Developed a dual-output deep learning model using 1D-CNN, BiLSTM, and Multi-Head Attention to predict both RUL and early fault risk.
2. Implemented a FedAvg federated learning baseline on the C-MAPSS dataset, where each client trained locally and only model weights were shared with the central server.
3. Explored three research directions: RQ2 using fault-aware aggregation for clients with fewer fault examples, RQ4 using concept drift injection with AUPRC-based detection and recovery, and RQ5 by studying how Non-IID validation can bias cross-client scoring and aggregation weights.

Some of these research question experiments are preliminary, but they provide useful findings and future directions for improving federated aircraft health monitoring.

2. Dataset and Initial Analysis

This project uses the C-MAPSS turbofan engine degradation dataset from *IEEEDataPort* [1]. It contains simulated run-to-failure data of turbofan engines. Each row represents one engine at one cycle and includes the engine ID, cycle number, three operational settings, and 21 sensor measurements. The dataset has four subsets: FD001, FD002, FD003, and FD004. In this project, I treated these four subsets as four different airline clients in the federated learning setup. The subsets have different operating conditions and fault modes, so they represent a *Non-IID* client setting. **Table 1** presents the dataset overview.

Table 1. C-MAPSS dataset overview

Subset	Operating Conditions	Fault Modes	Difficulty	Role in FL Setup
FD001	1	1	Easiest	Client 1
FD002	6	1	Harder	Client 2
FD003	1	2	Medium	Client 3
FD004	6	2	Hardest	Client 4

Initial Analysis and Visualization (Try-1): In Try-1, I performed exploratory data analysis to understand the structure of the C-MAPSS dataset before building the models. **Figures 1–5** show the engine lifetime distribution, sensor variance, sensor-RUL correlation, healthy versus near-failure sensor behavior, and cross-subset comparison across FD001–FD004. First, I checked the engine lifetime distribution and found that different engines run for different numbers of cycles before failure. Because of this, I used 30-cycle sliding windows instead of treating every engine as one fixed-length sequence. I also analyzed the sensor values to see which sensors were useful for degradation prediction. Some sensors had almost no variation, such as s1, s5, s10, s16, s18, and s19, so these sensors were removed from later experiments. I also checked the correlation between sensors and RUL. Sensors such as s11, s12, s15, s20, and s21 showed clearer changes as the engine moved closer to failure. I compared healthy and near-failure engine behavior and found that some sensor patterns changed noticeably near the failure stage. I also checked the early fault label distribution and observed class imbalance. For example, only about 18% of FD001 samples were fault-positive. Finally, I compared FD001–FD004 and found that the subsets had different fault ratios,

average lifetimes, operating conditions, and fault modes. This confirmed that the clients are Non-IID, which makes the dataset suitable for federated learning.

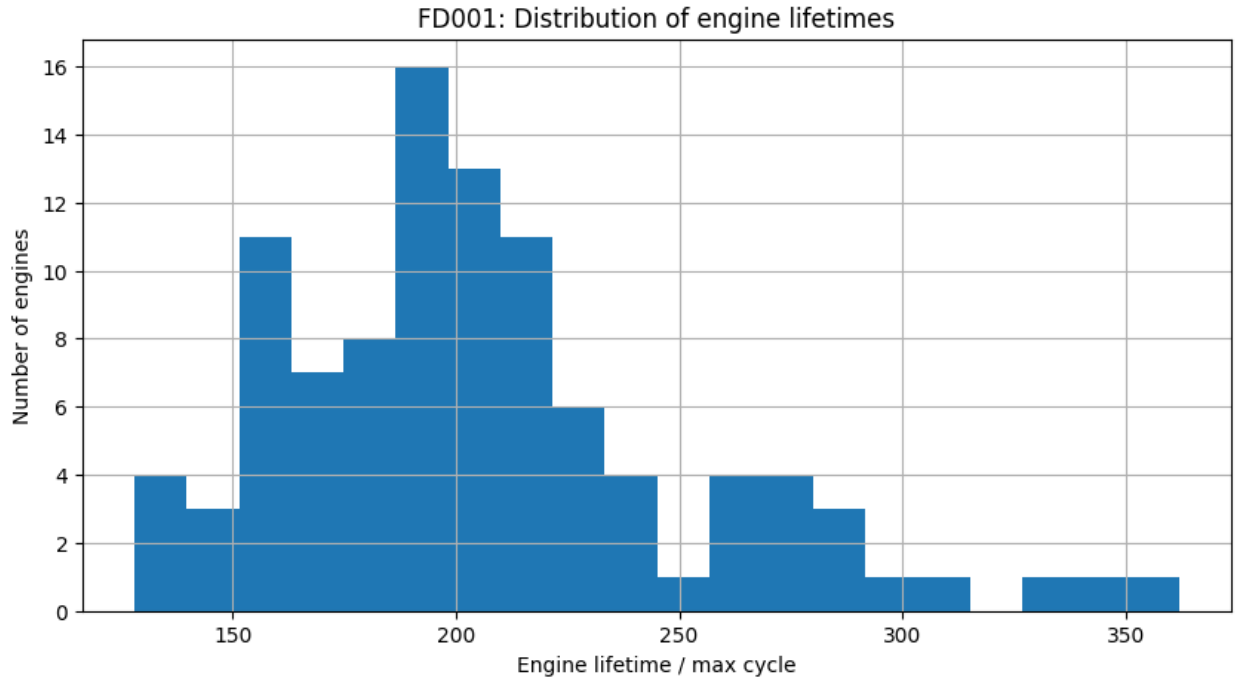


Figure 1. Engine lifetime distribution: This figure shows that engines have different operating lifetimes before failure, which supports using sliding windows instead of fixed-length engine sequences.

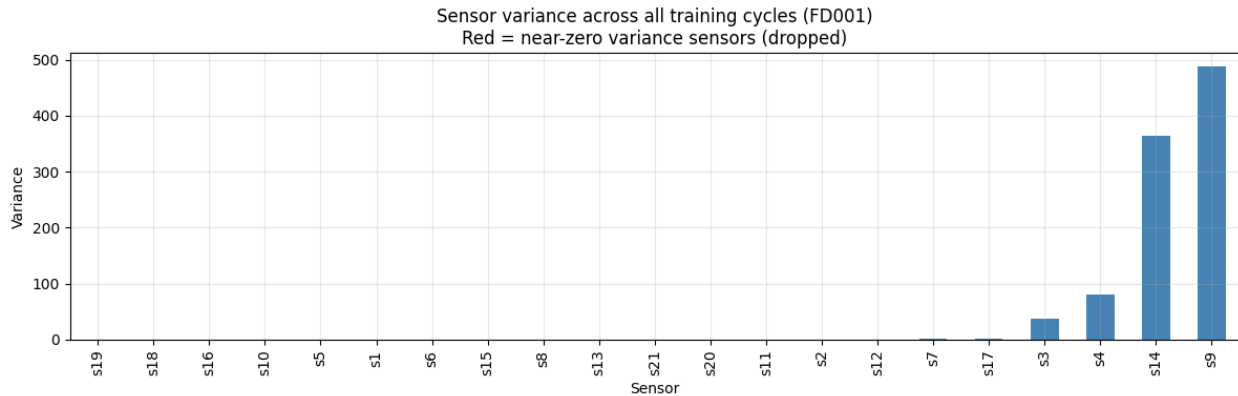


Figure 2. Sensor variance analysis: This figure shows the variation of each sensor. Sensors with very low variance, such as s1, s5, s10, s16, s18, and s19, were removed because they provided limited degradation information

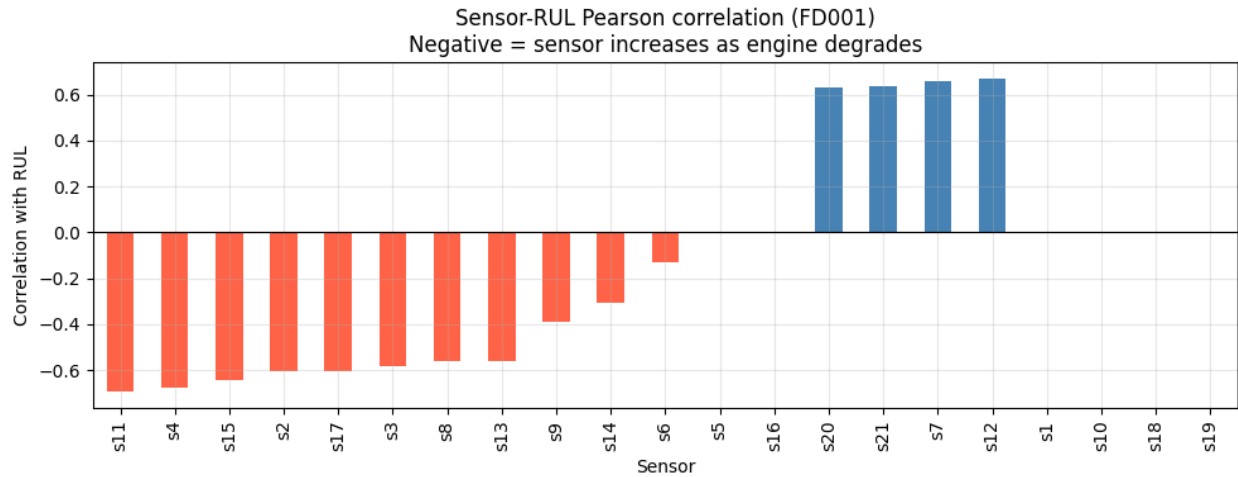


Figure 3. Sensor-RUL correlation: This figure shows the relationship between each sensor and Remaining Useful Life. Sensors such as s11, s12, s15, s20, and s21 showed stronger degradation-related patterns.

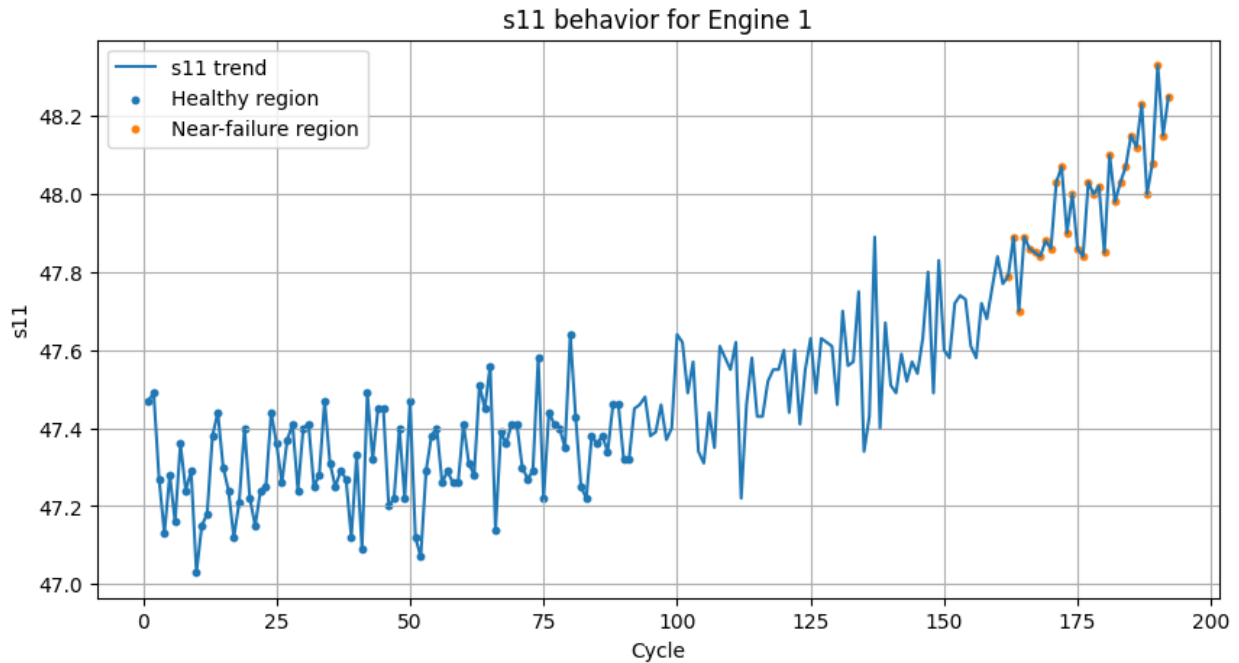


Figure 4. Healthy versus near-failure sensor comparison: This figure compares sensor behavior during healthy and near-failure stages. It shows that some sensors change clearly when the engine gets closer to failure.

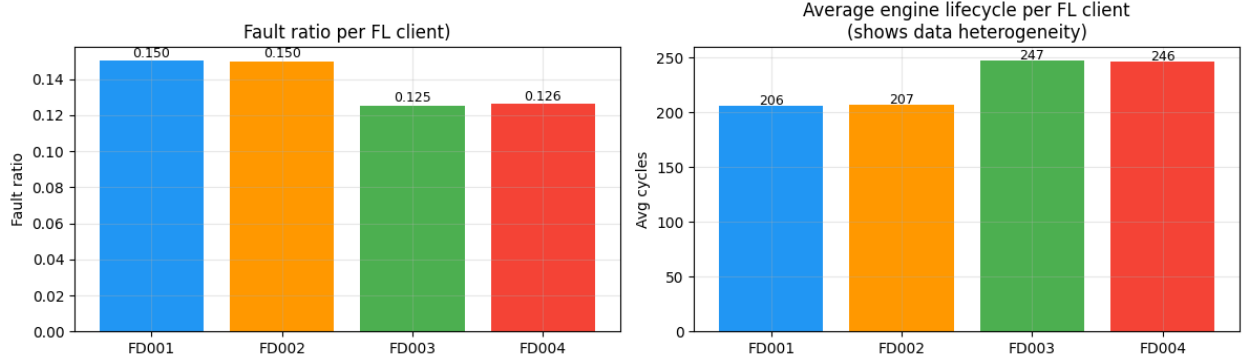


Figure 5. Cross-subset comparison of FD001–FD004: This figure compares the four C-MAPSS subsets using fault ratio, engine lifetime, and operating-condition differences. It shows that the clients are Non-IID, which makes the dataset suitable for federated learning.

3. Preprocessing Pipeline

The raw C-MAPSS data was processed before model training. First, the RUL label was calculated as the difference between the maximum engine cycle and the current cycle:

$$RUL = \max engine\ cycle - current\ cycle$$

To avoid very large RUL values dominating training, the RUL value was capped at 125 cycles.

For fault detection, a binary early fault label was created using the following rule [2]:

$$early\ fault = 1\ if\ RUL \leq 30$$

The data was split by engine ID instead of by rows to avoid data leakage. Sensor values were normalized using only the training data. Then, 30-cycle sliding windows were created, and the label of each window was taken from the final cycle of that window. For FD002 and FD004, cluster-based normalization was used because these subsets have multiple operating conditions. This helped normalize sensor values separately under different operating regimes. **Figure 6** presents the preprocessing pipeline.

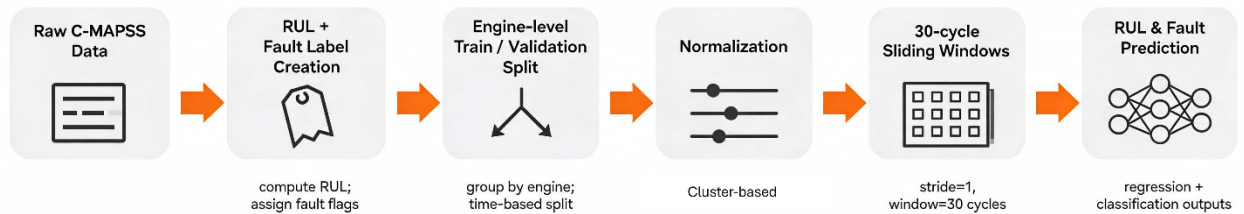


Figure 6. Preprocessing pipeline

4. Centralized Model Development

Before implementing federated learning, I first tested centralized models to understand whether the model could learn RUL and fault patterns from the data. After that, the improved dual-output model was used inside the federated learning setup.

4.1. Simple 1D-CNN Baseline

I first trained a simple centralized dual-output 1D-CNN model on FD001. **Figure 7** presents the 1D-CNN model architecture. **Table 2** presents the output on FD001. The model used 30-cycle sensor windows to predict both RUL and early fault risk. This baseline was used to check whether the model could learn basic degradation patterns from the C-MAPSS data. The model achieved reasonable RUL prediction, with a last-window MAE of 15.50 and R^2 of 0.753. However, the fault detection performance was weaker. The fault recall was only 0.560, meaning the model missed many near-failure cases. This happened mainly because the fault-positive samples were much fewer than the healthy samples. Therefore, the next step was to improve the model architecture and handle class imbalance more carefully.

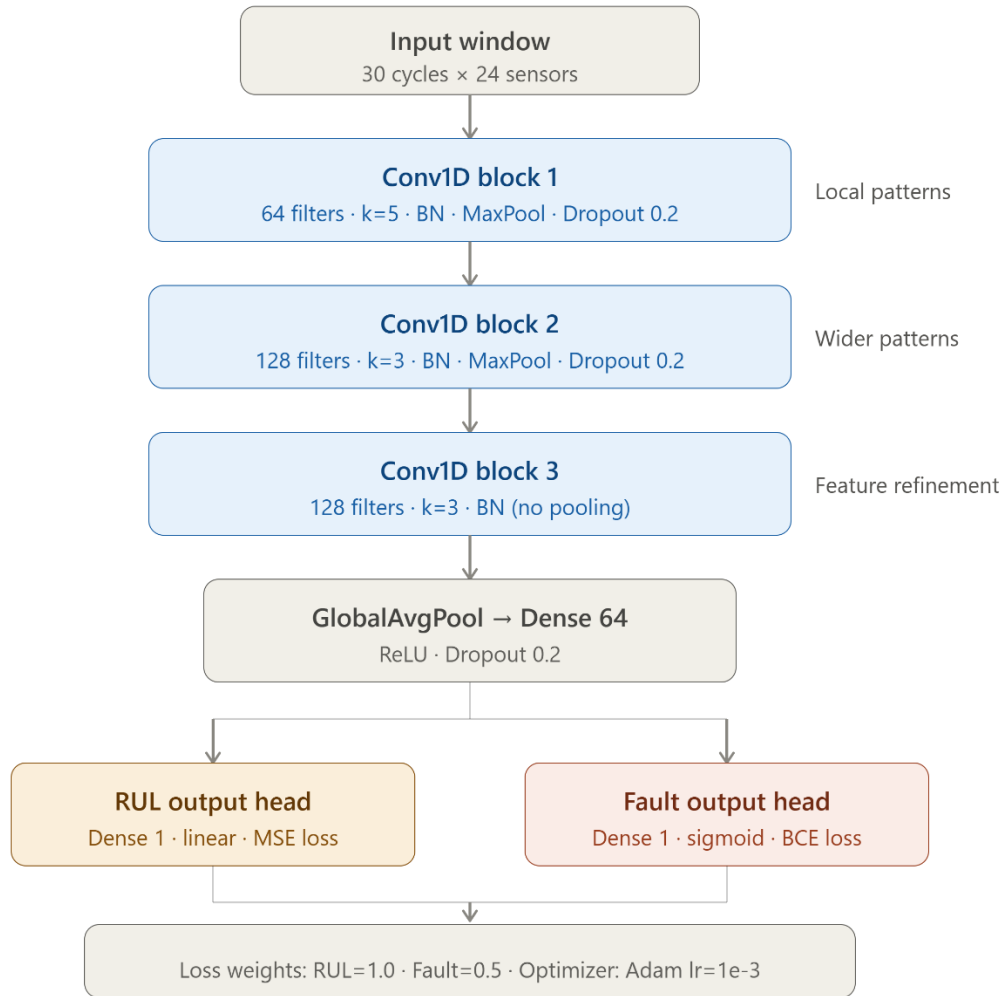


Figure 7. 1D-CNN architecture

Table 2. 1D-CNN result on FD001

Metric	All windows	Last window per engine
RUL MAE	13.08	15.50
RUL RMSE	17.80	19.91
RUL R^2	0.642	0.753
Fault Recall	0.539	0.560
Fault F1	0.645	0.700
AUPRC	0.792	0.938
AUROC	0.989	0.979

4.2. Improved 1D-CNN + BiLSTM + Multi-Head Attention Model

After the simple 1D-CNN baseline, I improved the model by adding BiLSTM and Multi-Head Attention. The 1D-CNN layers capture local sensor patterns inside each 30-cycle window, while the BiLSTM captures longer temporal degradation changes across the sequence. After that, Multi-Head Attention helps the model focus on the most important cycles in the window, especially the cycles closer to failure. I also removed six low-variance sensors, s1, s5, s10, s16, s18, and s19, because they did not provide useful degradation information. To improve early fault detection, I used weighted binary cross-entropy so that fault-positive samples received more importance during training. The overall model architecture is shown in **Figure 8**.

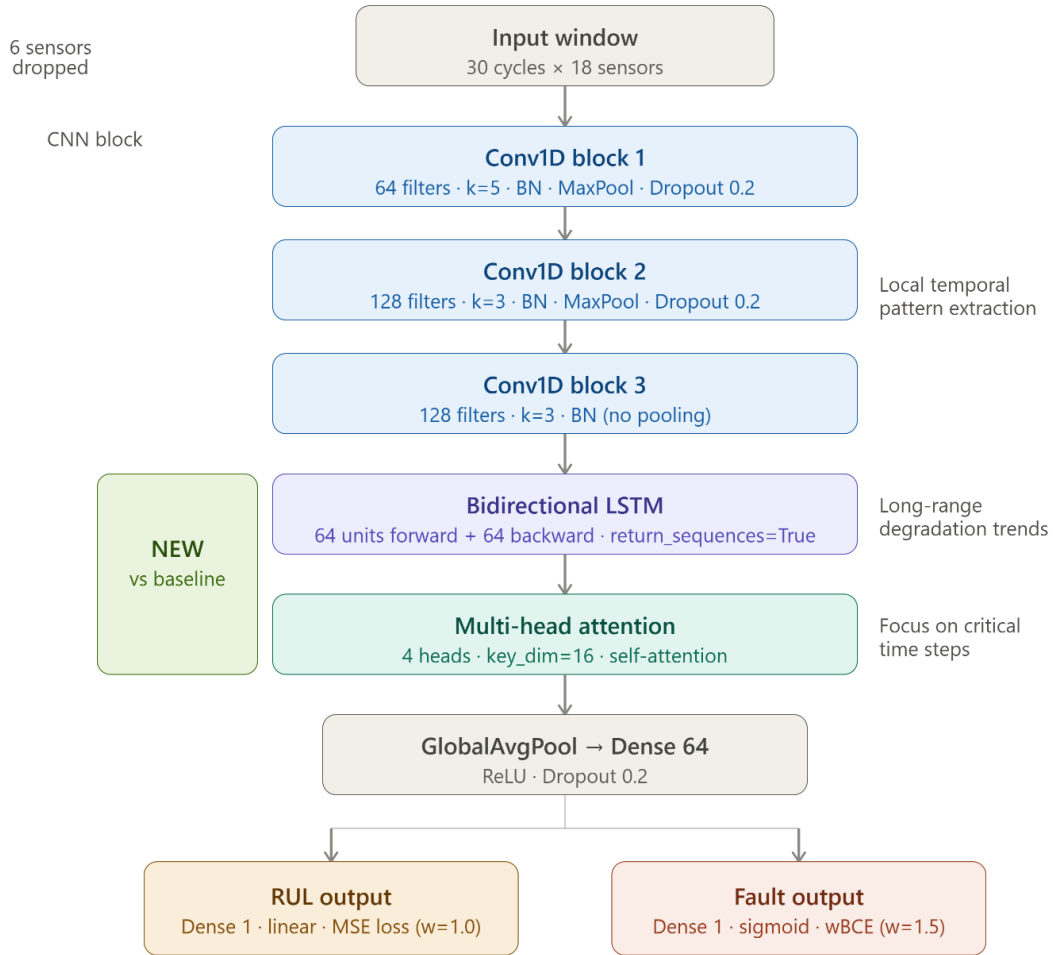


Figure 8. Improved dual-output model architecture

The model uses 1D-CNN layers to extract local sensor patterns, BiLSTM to capture temporal degradation behavior, and Multi-Head Attention to focus on important time steps. The final shared representation is used for two outputs: RUL regression and early fault classification.

The results of the improved centralized model are shown in **Table 3**. Compared with the simple 1D-CNN baseline result, the improved model gave much better fault detection performance. On FD001, fault recall increased from 0.560 to 0.920, and AUPRC improved from 0.938 to 0.977. The model also performed well

across the other subsets, reaching 1.000 recall on FD002, FD003, and FD004. These results show that adding BiLSTM, attention, sensor filtering, and weighted loss helped the model learn degradation and fault patterns more effectively before moving to the federated learning setup. Confusion matrices for all FD subsets are present in **Figure 9**.

Table 3. Improved centralized model (1D-CNN + BiLSTM + Multi-Head Attention Model) results using last-window-per-engine evaluation

Dataset	RUL MAE	R ²	Recall	F1	AUPRC	AUROC
FD001	10.98	0.850	0.920	0.885	0.977	0.992
FD002	14.70	0.814	1.000	0.891	0.982	0.995
FD003	14.16	0.739	1.000	0.851	0.959	0.989
FD004	16.81	0.765	1.000	0.848	0.956	0.987

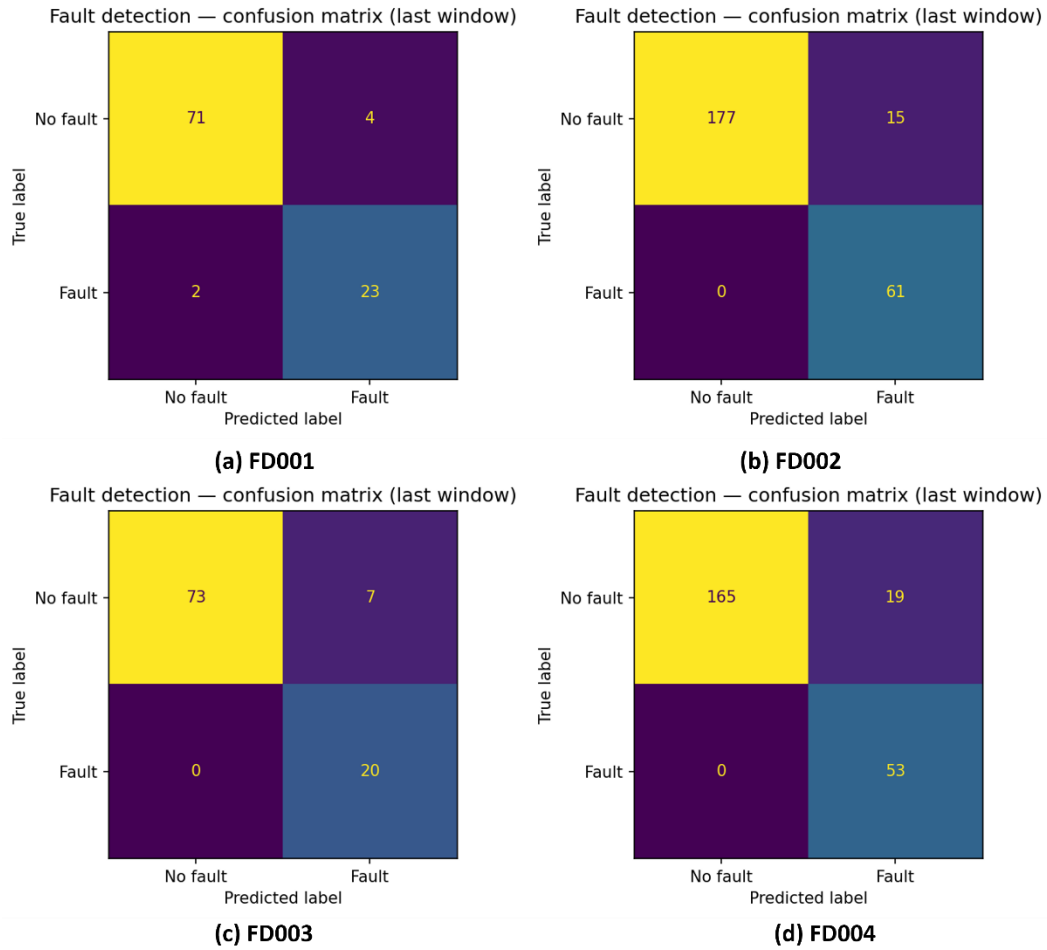


Figure 9. Confusion matrices for all FD subsets

5. Federated Learning Baseline

5.1. FedAvg Setup

After developing the improved centralized model, I used the same 1D-CNN + BiLSTM + Multi-Head Attention dual-output model as the local model inside the federated learning setup. The purpose of this

experiment was to implement the main federated learning baseline for aircraft engine health monitoring. In this setup, the four C-MAPSS subsets, FD001, FD002, FD003, and FD004, were treated as four separate airline clients. Each client kept its own local data and trained a copy of the model locally. The raw sensor data was never shared with the server. After local training, only the updated model weights were sent to the central server. The central server used FedAvg aggregation to combine the client models. In FedAvg, each client's model weights are averaged based on the number of training windows available for that client. Each client trained for 5 local epochs in each federated round, and the full process was repeated for 20 federated rounds. After training, the final global model was evaluated separately on each client's test data for both RUL prediction and early fault detection. The overall federated learning pipeline is shown in **Figure 11**.

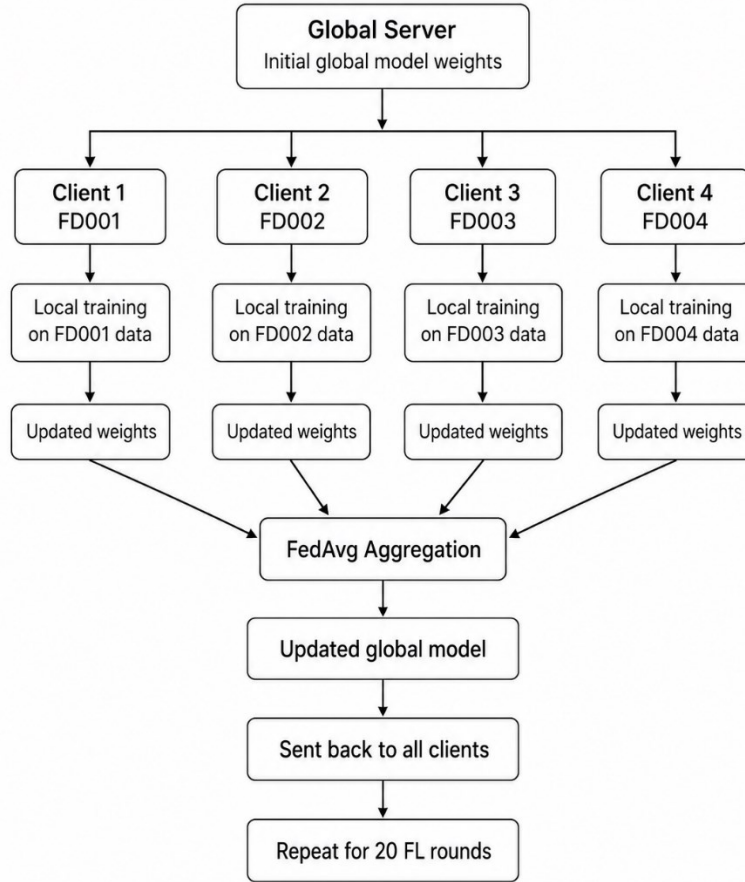


Figure 10. Federated learning pipeline diagram

5.2. FedAvg Results

The FedAvg baseline was trained for 20 federated rounds, with each client training locally for 5 epochs per round. The final global model was evaluated separately on each client. **Table 4** presents only the last-window-per-engine result, because it gives one final prediction for each test engine and is more suitable for RUL evaluation.

Table 4. FedAvg baseline results using last-window-per-engine evaluation

Client	RUL MAE	RUL RMSE	R ²	Accuracy	Precision	Recall	F1	AUPRC	AUROC
FD001	13.54	17.86	0.801	0.950	0.917	0.880	0.898	0.981	0.993

FD002	14.05	18.66	0.810	0.953	0.930	0.869	0.898	0.970	0.989
FD003	15.69	21.23	0.706	0.940	0.818	0.900	0.857	0.960	0.989
FD004	14.61	19.52	0.792	0.933	0.814	0.906	0.857	0.938	0.983

The FedAvg baseline achieved stable performance across all four clients. The last-window RUL MAE ranged from 13.54 to 15.69 cycles, showing that the global model could predict remaining life reasonably well across different C-MAPSS subsets. Fault detection performance was also strong, with recall between 0.869 and 0.906 and F1 between 0.857 and 0.898. FD001 and FD002 achieved the best overall results, with higher R^2 and AUPRC values. FD003 had the lowest R^2 , and FD004 had the lowest AUPRC, which suggests that these clients were more difficult due to different degradation patterns and operating conditions. Overall, this result shows that the federated baseline was working properly and can be used as the reference point for the later RQ2, RQ4, and RQ5 experiments. The FedAvg results are also shown in **Figure 11** and **Figure 12**. Figure 11 (fedavg_final_metrics) compares the final performance of all four clients. Figure 11 (fedavg_val_loss_per_round) shows the validation loss across 20 FL rounds and helps confirm that the federated training process was stable.

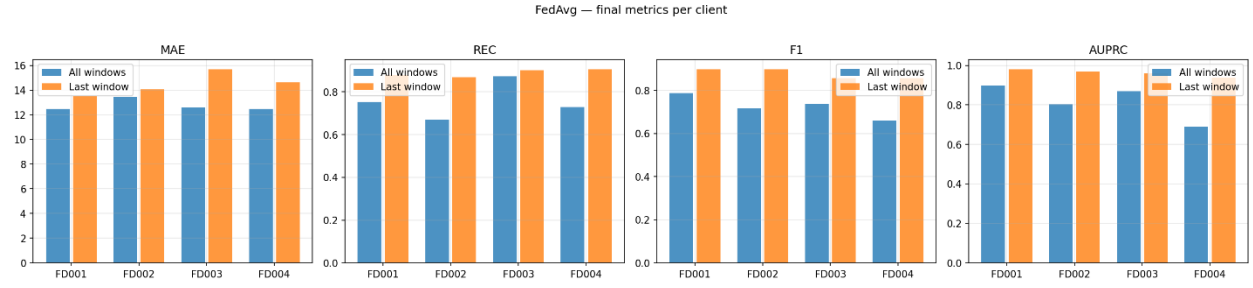


Figure 11. Final FedAvg performance across clients

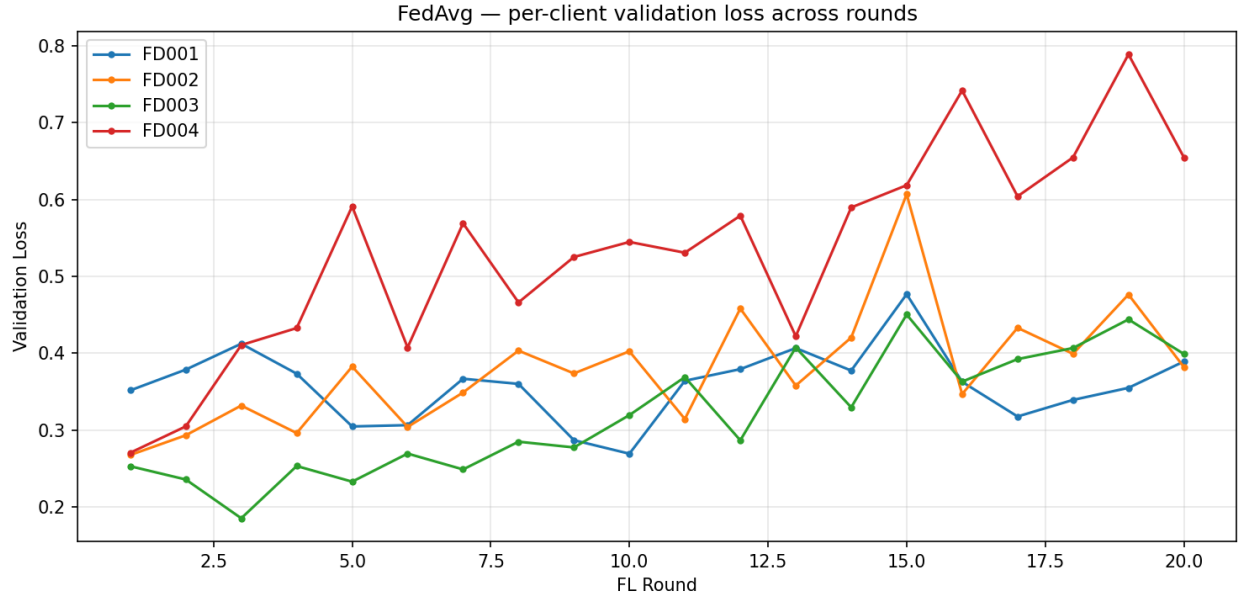


Figure 12. FedAvg validation loss across rounds

6. Research Question Experiments

After completing the FedAvg baseline, I explored three research questions: RQ2, RQ4, and RQ5. These experiments were used to test class imbalance, concept drift, and Non-IID validation bias in federated aircraft health monitoring.

6.1. RQ2: Class Imbalance Across Clients

To explore RQ2, I tested a fault-aware aggregation method. The same 1D-CNN + BiLSTM + Multi-Head Attention dual-output model from the FedAvg baseline was used, so the model architecture was not changed. The main change was in the aggregation strategy. In normal FedAvg, clients are usually weighted based on the number of local training samples. However, this can weaken the learning signal from clients that have fewer fault-positive examples. To reduce this problem, I calculated each client’s fault ratio and used it during aggregation. The experiment was trained for 25 FL rounds, with 8 local epochs and $\alpha = 0.5$. The fault ratio table (**Table 5**) shows that FD003 and FD004 had slightly fewer fault-positive samples than FD001 and FD002. The main comparison is shown in **Table 6** using last-window-per-engine results.

Table 5. Fault ratio per client

Client	Fault Ratio
FD001	0.1741
FD002	0.1723
FD003	0.1402
FD004	0.1417

Table 6. RQ2 fault-aware aggregation results compared with FedAvg baseline

Client	Baseline Recall	RQ2 Recall	Baseline F1	RQ2 F1	Baseline AUPRC	RQ2 AUPRC
FD001	0.880	0.880	0.898	0.898	0.981	0.961
FD002	0.869	0.869	0.898	0.876	0.970	0.971
FD003	0.900	1.000	0.857	0.909	0.960	0.977
FD004	0.906	0.943	0.857	0.833	0.938	0.948

The fault-aware aggregation improved the lower-fault clients more clearly. For FD003, recall increased from 0.900 to 1.000, F1 improved from 0.857 to 0.909, and AUPRC improved from 0.960 to 0.977. For FD004, recall increased from 0.906 to 0.943, and AUPRC improved from 0.938 to 0.948, although F1 decreased slightly. FD001 and FD002 showed little or no improvement. These results suggest that the method can help clients with weaker fault signals, but the improvement is not consistent for all clients. As shown in **Figure 13** and **Figure 14**, the result indicates that fault-aware aggregation is promising, but not yet a complete solution for RQ2. Since the fault-ratio difference across clients was relatively small, this experiment should be viewed as an initial validation. A more rigorous evaluation would require stronger client-level imbalance to test whether the method can consistently protect rare fault signals.

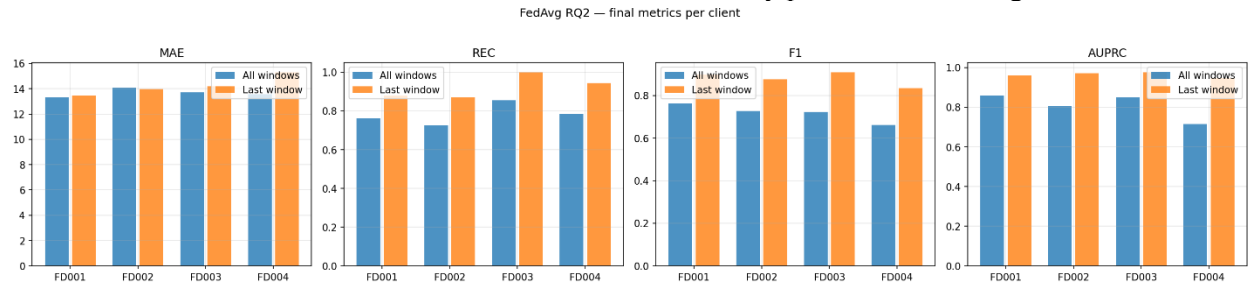


Figure 13. RQ2 final metrics across clients

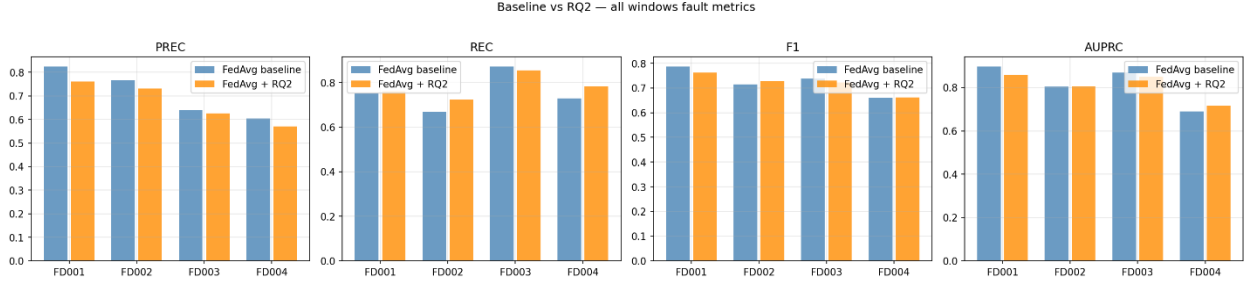


Figure 14. RQ2 comparison with FedAvg baseline

6.2. RQ4: Concept Drift

To explore RQ4, I simulated concept drift in the federated setting. The same dual-output model from the FedAvg baseline was used, but a drift injection and recovery mechanism were added. FD001 was selected as the drifted client, and an FD002-style shift was injected at federated round 13. AUPRC was monitored as the main drift metric. If the AUPRC dropped by more than 0.03, the client was flagged for drift, and a recovery step was applied using a higher learning rate of 0.003.

Table 7. RQ4 concept drift detection and recovery summary

Item	Result
Drift client	FD001
Drift round	13
Detection round	17
Detection delay	4 rounds
Pre-drift AUPRC	0.9731
Min post-drift AUPRC	0.9327
AUPRC drop	0.0403
After 10 recovery rounds	0.9578
Recovery gain	0.0251
Fully recovered	No

As shown in **Table 7**, the injected drift reduced FD001’s AUPRC from 0.9731 to 0.9327, showing that the distribution shift affected fault detection performance. The detector identified the drift at round 17, which is four rounds after the drift was injected. After recovery, AUPRC improved to 0.9578, but it did not fully return to the pre-drift level. Therefore, this experiment partially answers RQ4: the drift was detected and recovery helped, but full recovery was not achieved. Also, some non-drifted clients were flagged early, so the drift detector needs further improvement. **Figure 15** further shows how AUPRC changed during training and highlights the effect of drift and recovery. **Figure 16** focuses on FD001 and shows the AUPRC drop after drift injection and the partial recovery afterward

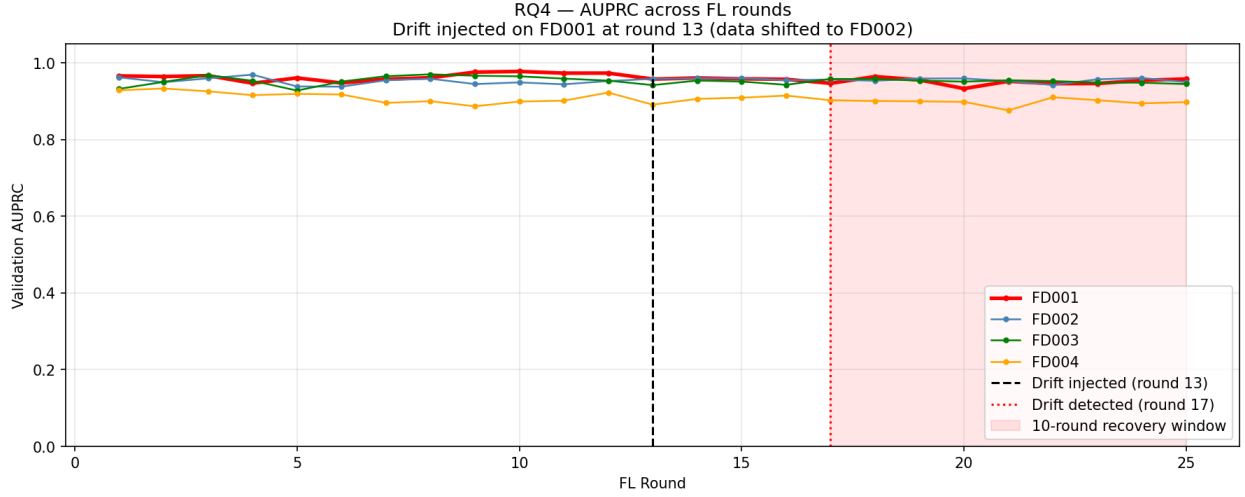


Figure 15. RQ4 AUPRC trend across federated rounds

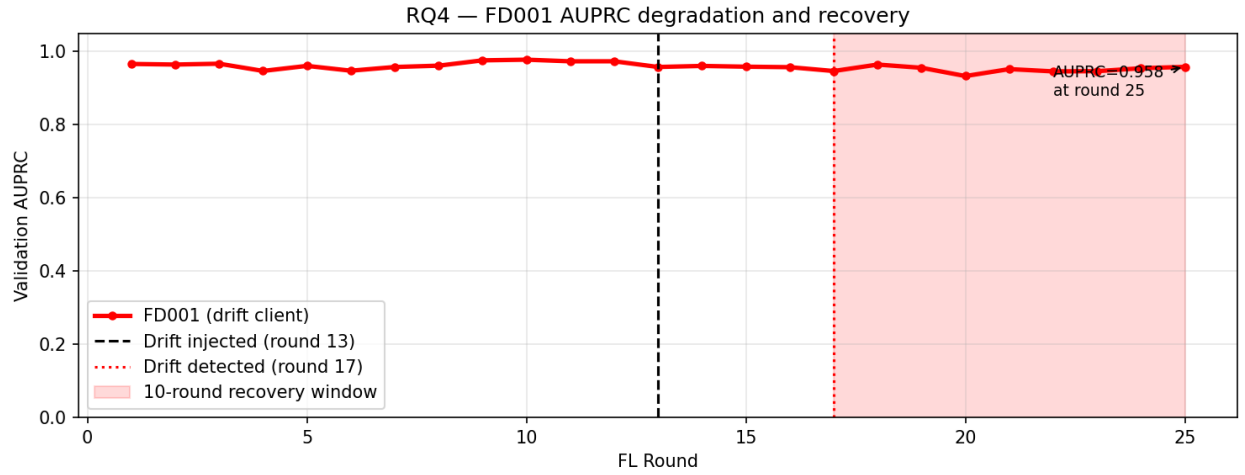


Figure 16. RQ4 drift client zoom view

6.3. RQ5: Validation Under Non-IID

RQ5 was another important research direction explored in this work. This experiment tested whether cross-client validation can unfairly score models when client data are Non-IID. The same trained client models from the FedAvg setting were used. Each client model was evaluated on every other client’s data, and AUPRC was used as the validation score. Then, MMD was used to estimate distributional distance between clients, and standard validation-based aggregation weights were compared with corrected weights. The cross-evaluation results in **Table 8** and **Figure 17** show that model performance depends strongly on which client’s data is used for validation. A lower score does not always mean the model is poor; it may simply mean the validation client has a different operating distribution.

Table 8. Cross-evaluation AUPRC matrix for RQ5

Data	FD001	FD002	FD003	FD004
FD001	0.9058	0.7813	0.8512	0.6175
FD002	0.8858	0.8239	0.7019	0.6803
FD003	0.8827	0.7549	0.8789	0.6453
FD004	0.8314	0.7351	0.7125	0.6165

The MMD distance matrix, shown in **Figure 18**, was used to measure distributional mismatch between clients. After accounting for this mismatch, the aggregation weights changed, as shown in **Table 9** and **Figure 19**. The largest change was for FD004, whose weight increased from 0.2353 to 0.2431. This suggests that standard validation-based scoring may have underweighted FD004 because its data distribution was different, not because its model was necessarily less useful.

Table 9. Standard and distribution-corrected aggregation weights

Client	Standard Weight	Corrected Weight	Change
FD001	0.2565	0.2522	-0.0043
FD002	0.2513	0.2505	-0.0007
FD003	0.2569	0.2542	-0.0027
FD004	0.2353	0.2431	+0.0078

Overall, this experiment strongly supports the main idea of RQ5: under Non-IID data, cross-client validation can punish distributional difference instead of true model quality. The largest self-vs-cross validation gap was 0.1558 on FD001, and the corrected weighting changed client contributions, especially for FD004. However, this experiment still partially answers RQ5 because the corrected weights were not yet used in a full retraining experiment to compare final per-client RUL MAE/R². A complete next step would be to apply these corrected weights during aggregation and measure whether per-operator RUL accuracy improves.

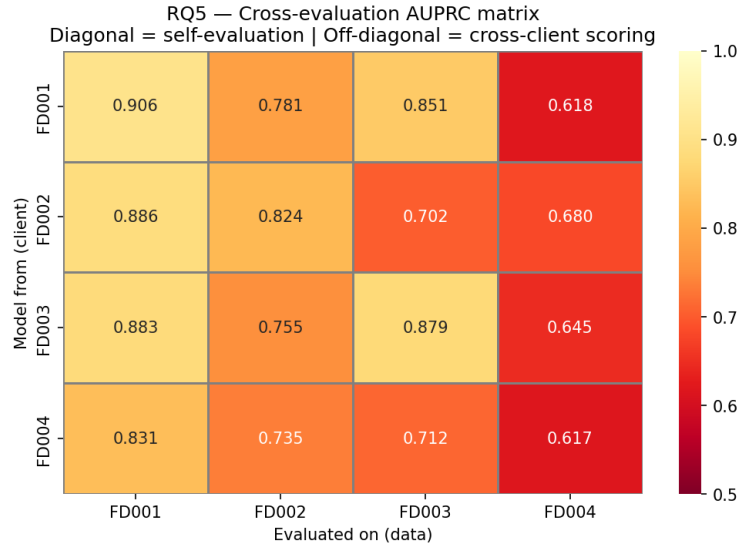


Figure 17. RQ5 cross-evaluation AUPRC matrix

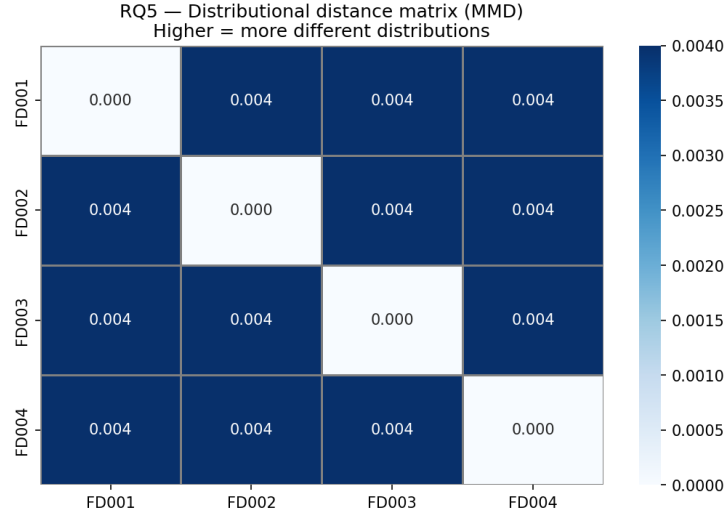


Figure 18. RQ5 MMD distributional distance matrix

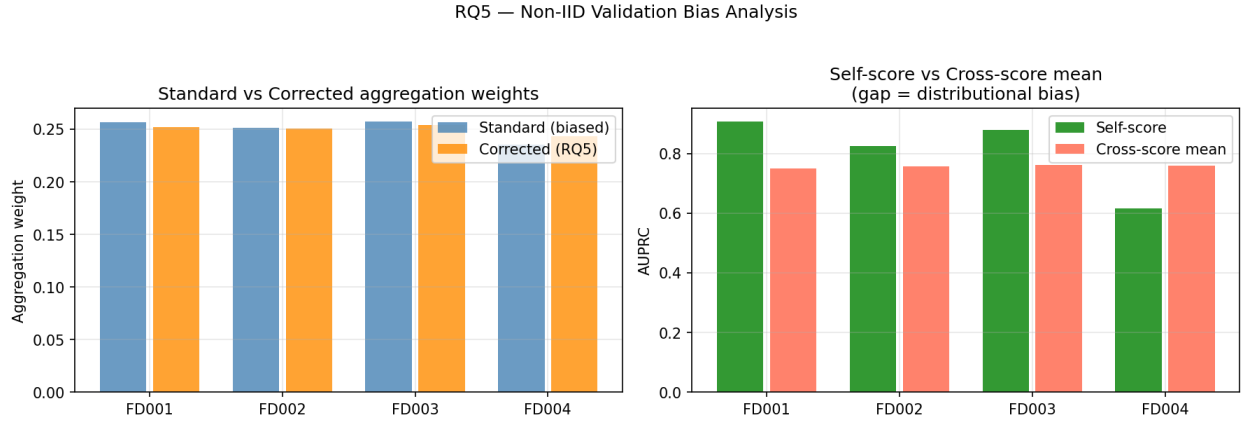


Figure 19. RQ5 standard vs corrected aggregation weights

7. Limitations and Future Work

This work has some limitations that can be improved in future research. First, the experiments were based on the simulated C-MAPSS dataset, so future work should test the pipeline on N-CMAPSS or other real aviation datasets. Second, the class imbalance in RQ2 was mild, so stronger imbalance should be created by reducing fault-positive samples for selected clients. Third, the RQ4 drift detector showed partial recovery but also produced false positives, so a more reliable drift detection method is needed. Fourth, RQ5 showed that Non-IID validation changes aggregation weights, but the corrected weights should be applied in full federated training and compared using final RUL MAE and R^2 . Finally, future work can add privacy and security testing, such as membership inference attacks, poisoning attacks, or secure aggregation.

8. Conclusion

In this project, I completed the main federated learning baseline using FedAvg with four C-MAPSS clients: FD001, FD002, FD003, and FD004. The final model was designed to predict both Remaining Useful Life (RUL) and early fault risk, which satisfies the main baseline requirement of the task. I also explored three research questions: RQ2 on class imbalance, RQ4 on concept drift, and RQ5 on Non-IID validation bias. The RQ2 and RQ4 experiments showed useful but partial results, while RQ5 showed that cross-client validation can be affected by distribution mismatch and can change aggregation weights. Overall, these experiments provide a working baseline and useful future directions for improving federated aircraft health monitoring.

References

- [1] A. Saxena, K. Goebel, D. Simon, N. Eklund, Damage propagation modeling for aircraft engine run-to-failure simulation, in: 2008 International Conference on Prognostics and Health Management, IEEE, 2008: pp. 1–9. <https://doi.org/10.1109/PHM.2008.4711414>.
- [2] H. Özcan, Predictive maintenance in aircraft engine maintenance using the C-MAPSS dataset: performance comparison and evaluation of machine learning classification algorithms, *Artificial Intelligence for Engineering Design, Analysis and Manufacturing* 40 (2026) e4. <https://doi.org/10.1017/S0890060426100249>.